DEEPMINDQ

Enterprise Revenue Intelligence Operating System

Definitive Architecture Action Plan

Milestone-by-Milestone Implementation Roadmap
with File-Level Deliverables Mapped Against Existing Code

Version 1.0 | Architecture Debate Document | Not for Execution
18 Milestones | 8 Layers | 15 Missing Modules | ~35% Unbuilt

Table of Contents

Chapter 1: Executive Summary

DeepMindQ is an Enterprise Revenue Intelligence Operating System, not a CRM or SaaS product. The current codebase represents a substantial foundation with 166 API routes, 7 composable AI engines, 96 Prisma models, and a working intelligence pipeline. However, honest assessment against the full 8-layer target architecture reveals that approximately 35% of the vision remains unbuilt, 15% is built but requires repair, and 15% is partially implemented. Only 25% of the target architecture is fully built and working, with an additional 10% in a functional but unoptimized state.

This action plan provides a definitive, milestone-by-milestone roadmap from the current state to the full target architecture. Every milestone includes specific file-level deliverables: which existing files are modified and how, which new files are created, what the output looks like, and what acceptance criteria must be met. The plan is organized around 8 architectural layers with 18 milestones, covering type safety repair, intelligence engine consolidation, governance wiring, security hardening, enterprise readiness, and UI/UX modernization.

The plan is designed for debate and review. No code execution should begin until all stakeholders agree on the milestone sequence, dependency chain, and acceptance criteria. The dependency chain is explicit: Milestones 1-4 are foundational and must complete before parallel work on Milestones 5-12 can begin. Milestones 13-18 represent the final integration and polish layer.

Metric	Current State	Target State
Total API Routes	166 (working)	~200 (with enterprise)
AI Engines (Working)	7 (Architecture 1)	15+ (all architectures merged)
TypeScript Errors	0 (via @ts-nocheck)	0 (real type safety)
@ts-nocheck Files	24 files	0 files
Tests Passing	829/829	1000+/1000+
DB Session-Validated Routes	8 (4%)	All protected routes
Governance-Wired Engines	1 of 7	All engines
Screen Components	80 screens, 63 view IDs	~20 screens, consolidated
Prisma Models	96 models, 19 enums	100+ models with tenant support

Table 1: Key Metrics - Current vs Target

Chapter 2: Current State Assessment

2.1 What Is Built and Working (25%)

The core IP of DeepMindQ resides in the 7 composable engines under src/lib/engines/, totaling 4,574 lines of well-structured, typed, tested TypeScript. These engines form Architecture 1 (Phase B Composable Engines) and represent the production-ready foundation upon which everything else must be built. The ModelRouter provides tiered LLM routing with automatic fallback, token

accounting, and a non-throwing design contract. The GroundingEngine implements evidence-chain construction with 15 hallucination prevention rules. The RetrievalEngine handles local embeddings for knowledge search. The SynthesisEngine combines multiple intelligence signals. The ScoringEngine provides 9-dimension scoring. The ActionEngine generates prioritized action plans. The ConversationEngine handles multi-turn conversation planning and context management.

Beyond the engines, the Prisma schema is extensive at 2,890 lines with 96 models and 19 enums. Critically, the schema already includes models for the Layer 3 (Enterprise Reasoning) architecture: ReasoningContext, ReasoningStep, AgentOrchestration, LearningEvent, FusionResult, KnowledgeDocument, and AICallLog all exist. The 166 API routes are real end-to-end endpoints, not stubs. The proxy.ts provides cookie-level authentication on all /api/* routes with CSRF timing-safe comparison and rate limiting. The quality gates are green: TSC 0 errors (achieved via @ts-nocheck), 829/829 tests passing, ESLint 0 errors, and build success.

Engine	File	Lines	Status
ModelRouter	src/lib/engines/model-router.ts	428	Working
GroundingEngine	src/lib/engines/grounding-engine.ts	579	Working
RetrievalEngine	src/lib/engines/retrieval-engine.ts	484	Working
SynthesisEngine	src/lib/engines/synthesis-engine.ts	675	Working
ScoringEngine	src/lib/engines/scoring-engine.ts	811	Working
ActionEngine	src/lib/engines/action-engine.ts	691	Working
ConversationEngine	src/lib/engines/conversation-engine.ts	829	Working

Table 2: Architecture 1 - Working Composable Engines

2.2 What Is Built but Needs Repair (15%)

The most significant repair target is the Layer 3 Enterprise Reasoning architecture. Four files totaling ~1,560 lines are marked with @ts-nocheck, but investigation reveals that the Prisma models they reference (ReasoningContext, ReasoningStep, AgentOrchestration, LearningEvent, FusionResult) actually exist in the schema. This means these engines are likely 70-80% functional but have never been type-validated. The enterprise-reasoning-engine.ts (667 lines) implements a 30-step cumulative reasoning chain. The multi-agent-orchestrator.ts (413 lines) coordinates 10 specialist agents. The fusion-engine.ts (276 lines) fuses external signals with internal capabilities. The continuous-learning-loop.ts (204 lines) learns from wins, losses, and feedback.

Additional repair targets include 20 other @ts-nocheck files spread across intelligence sources, connectors, API routes, and utility modules. The 156 "as any" type escapes across 36 source files represent systematic type safety erosion that must be addressed engine by engine. The 328 instances of text-[10px] across 22 files, while rendered at 11px via CSS override, represent code hygiene debt. The revenue-intelligence/account-scoring.ts (413 lines) is a legacy 5-dimension scorer that must be deprecated in favor of the hybrid scoring model (AccountPriorityScore + IntelligenceScore).

File	Lines	Category	Repair Estimate
src/lib/enterprise-reasoning-engine.ts	667	Layer 3 Core	High - remove @ts-nocheck, validate types

File	Lines	Category	Repair Estimate
src/lib/multi-agent-orchestrator.ts	413	Layer 3 Core	High - remove @ts-nocheck, validate types
src/lib/fusion-engine.ts	276	Layer 3 Core	Medium - remove @ts-nocheck, validate types
src/lib/continuous-learning-loop.ts	204	Layer 3 Core	Medium - remove @ts-nocheck, validate types
src/lib/knowledge-ingestion-pipeline.ts	-	Intelligence Source	Medium - Prisma model validation
src/lib/ai-cache-layer.ts	-	AI Infrastructure	Low - type validation
src/lib/ai-copilot/usage-tracker.ts	-	AI Copilot	Low - type validation
src/lib/ai-copilot/quality-gates.ts	-	AI Copilot	Low - type validation
+ 16 more files	-	Various	Low-Medium

Table 3: @ts-nocheck Files Requiring Repair

2.3 What Is Partially Built (25%)

The AI Governance layer (src/lib/ai-governance.ts, 1,105 lines) is substantial and well-designed with 30 generation-type configs, 15 hallucination prevention rules, and 4-domain freshness lifecycle. However, it is only wired into the GroundingEngine and 5 legacy files. The SynthesisEngine, ScoringEngine, ActionEngine, and ConversationEngine all make direct LLM calls through ModelRouter without passing through the governance layer. This is a critical gap: the governance framework exists but is bypassed by 5 of 7 composition engines.

The RBAC system (src/lib/rbac.ts, 85 lines) defines role-permission mappings for admin, manager, and sales_rep roles across 9 resource types. However, only 1 API route uses withApiMiddleware from auth-helpers.ts. The proxy.ts provides cookie-level auth on all routes, but only 8 routes (4%) perform DB-level session validation. The store.ts has 63 view IDs mapped to 80 screen components, indicating severe screen sprawl that needs consolidation to approximately 20 primary views. The 4 @ts-nocheck files in the Layer 3 architecture represent ~1,560 lines of code that is structurally complete but has never been type-validated.

2.4 What Is Not Built at All (35%)

The largest gap is in the Enterprise Platform Layer (Layer 8). There is no multi-tenant architecture: no tenantId on any Prisma model, no tenant isolation middleware, no role-based access control enforcement at the API level, no audit trail for sensitive operations (beyond basic auth failure logging), no data isolation per tenant, and no enterprise SSO integration (SAML/OIDC). The proxy.ts DEMO_MODE bypass at line 69 disables all authentication when the environment variable is set, which is a development convenience but must be removed for production.

Additional unbuilt modules include: the Intelligence Orchestration Layer (Layer 3) integration - the individual engines exist but are not wired together into a unified reasoning pipeline; the Governed AI Engine Layer (Layer 4) enforcement - governance exists but is not universally wired; the Business Intelligence Layer (Layer 5) real-time analytics and predictive modeling; the Data Intelligence Layer (Layer 6) automated data quality monitoring and enrichment; and the unified Experience Layer (Layer 2) with the Intelligence OS design system fully implemented. The target architecture specifies 15 missing enterprise modules across these 8 layers that must be designed, built, and integrated.

Chapter 3: Eight-Layer Architecture Gap Analysis

Layer	Name	Built %	Key Gaps	Priority
L1	Enterprise Users	30%	No SSO/SAML, no RBAC enforcement, DEMO_MODE bypass, 4% DB-validated routes	Critical
L2	AI Experience Layer	40%	80 screens (should be ~20), no side panel design, generic EmptyState, no progressive disclosure	High
L3	Intelligence Orchestration	50%	Layer 3 engines @ts-nocheck, no unified pipeline, orchestrator not wired to API routes	Critical
L4	Governed AI Engine	35%	Governance in 1/7 engines, no enforcement before LLM calls, cost governance unbuilt	Critical
L5	Business Intelligence	25%	No predictive modeling, no real-time analytics, legacy scoring not deprecated	High
L6	Data Intelligence	30%	Data quality modules exist but not integrated, no automated monitoring, no enrichment pipeline	Medium
L7	Data Foundation	55%	96 Prisma models but no tenantId, no migration to multi-tenant, no data isolation	High
L8	Enterprise Platform	15%	No multi-tenancy, no audit trail, no compliance framework, no enterprise deployment	Critical

Table 4: Eight-Layer Architecture Gap Summary

Chapter 4: Milestone-by-Milestone Action Plan

The following 18 milestones are organized into 5 tracks with explicit dependencies. Milestones 1-4 form the foundation track and must complete sequentially. Milestones 5-9 form the core intelligence track and can partially parallelize after M4. Milestones 10-12 form the enterprise readiness track. Milestones 13-15 form the experience track. Milestones 16-18 form the integration and polish track. Each milestone lists specific files to modify, files to create, the deliverable output, and acceptance criteria.

Milestone 1: Type Safety Foundation

Track: Foundation | Depends on: None | Estimated Effort: 2-3 days

Remove @ts-nocheck from the 4 Layer 3 core engines and validate all types against existing Prisma schema. These files reference ReasoningContext, ReasoningStep, AgentOrchestration, LearningEvent, and FusionResult models that DO exist in the schema. The @ts-nocheck annotations are stale and can be removed with targeted type fixes. This milestone unblocks all subsequent engine work.

Action	File(s)	Changes
Remove @ts-nocheck, validate types	src/lib/enterprise-reasoning-engine.ts	Remove line 1 @ts-nocheck. Fix any type mismatches against Prisma ReasoningContext/ReasoningStep models. Add proper type imports.
Remove @ts-nocheck, validate types	src/lib/multi-agent-orchestrator.ts	Remove line 1 @ts-nocheck. Fix types against AgentOrchestration model. Validate ModelRouter.complete() return types.
Remove @ts-nocheck, validate types	src/lib/fusion-engine.ts	Remove line 1 @ts-nocheck. Fix FusionResult type references. Validate all Prisma queries.
Remove @ts-nocheck, validate types	src/lib/continuous-learning-loop.ts	Remove line 1 @ts-nocheck. Fix LearningEvent type references. Validate RetrievalEngine usage.
Fix as any type escapes	src/lib/revenue-intelligence/signal-detector.ts (lines 228, 241)	Replace `as any` with proper typed interfaces.
Fix as any type escapes	src/lib/revenue-intelligence/brief-generator.ts (lines 99, 110, 114, 120, 232, 242, 294, 334, 343, 352)	Replace `as any` with proper Prisma select types and signal type unions.
Fix as any type escapes	src/lib/source-reliability.ts (line 15)	Replace `(db as any).evidenceSourceReliability` with proper db import.
Add targeted tests	tests/enterprise-reasoning.test.ts (NEW)	Create test file for reasoning engine type validation.
Add targeted tests	tests/multi-agent-orchestrator.test.ts (NEW)	Create test file for orchestrator type validation.

Table 5: Milestone 1 - File-Level Deliverables

Deliverable: 4 Layer 3 engines compile without @ts-nocheck. TSC passes with 0 errors. 2 new test files with type-validation tests. Build green.

Acceptance Criteria: (1) grep -r "@ts-nocheck" src/lib/enterprise-reasoning-engine.ts src/lib/multi-agent-orchestrator.ts src/lib/fusion-engine.ts src/lib/continuous-learning-loop.ts returns 0 matches. (2) TSC completes with 0 errors. (3) 829+ tests still pass. (4) ESLint 0 errors.

Milestone 2: AI Governance Wiring

Track: Foundation | Depends on: M1 | Estimated Effort: 2-3 days

Wire the AI Governance layer (src/lib/ai-governance.ts, 1,105 lines) into ALL 7 composition engines before every LLM call. Currently only GroundingEngine imports from ai-governance.ts (FRESHNESS_LIFECYCLE_DAYS). The other 6 engines call ModelRouter.complete() directly without governance checks. This milestone ensures every AI output passes through confidence gates, hallucination prevention, evidence grounding, and audit trail logging.

Action	File(s)	Changes
Wire governance before LLM calls	src/lib/engines/synthesis-engine.ts	Import AIGovernance. Add pre-generation governance check before ModelRouter.complete(). Log results to AICallLog.
Wire governance before LLM calls	src/lib/engines/scoring-engine.ts	Import AIGovernance. Add governance check with generationType="scoring". Reject if confidence below threshold.
Wire governance before LLM calls	src/lib/engines/action-engine.ts	Import AIGovernance. Add governance check with generationType="action_plan".
Wire governance before LLM calls	src/lib/engines/conversation-engine.ts	Import AIGovernance. Add governance check with generationType="conversation".

Action	File(s)	Changes
Wire governance before LLM calls	src/lib/engines/retrieval-engine.ts	Import AIGovernance. Add governance check for retrieval quality.
Wire governance into L3 engines	src/lib/enterprise-reasoning-engine.ts	Add governance checks in each of the 30 reasoning steps before LLM calls.
Wire governance into orchestrator	src/lib/multi-agent-orchestrator.ts	Add governance gate before agent execution.
Create governance integration tests	tests/governance-wiring.test.ts (NEW)	Test that all engines call governance before LLM. Verify rejection on low confidence.
Update ESLint governance rule	eslint-rules/no-ungoverned-llm.js	Update rule to flag any ModelRouter.complete() call not preceded by governance check.

Table 6: Milestone 2 - File-Level Deliverables

Deliverable: All 7+ engines call governance before LLM. ESLint rule enforces no ungoverned LLM calls. New governance wiring tests pass. AICallLog records all generations.

Acceptance Criteria: (1) grep for ModelRouter.complete() in engine files shows governance check in every calling function. (2) ESLint no-ungoverned-llm rule passes with 0 violations. (3) New tests pass. (4) AICallLog Prisma model receives records from all engines.

Milestone 3: Security Hardening

Track: Foundation | Depends on: M1 | Estimated Effort: 2-3 days

Hardening the authentication and authorization layer from 4% DB-validated routes to full coverage. Remove the DEMO_MODE bypass that disables all authentication. Extend withApiMiddleware usage from 1 route to all protected routes. Add RBAC enforcement at the API level using the existing rbac.ts definitions. This milestone addresses the most critical security gaps identified in the audit.

Action	File(s)	Changes
Remove DEMO_MODE bypass	src/proxy.ts (line 68-71)	Remove or guard the `if (NEXT_PUBLIC_DEMO_MODE === true) return response` block. Replace with env-gated dev-only bypass that requires explicit DEVELOPER_SESSION token.
Expand withApiMiddleware usage	src/lib/auth-helpers.ts	Enhance withApiMiddleware to accept role requirements. Add helper: withAdminApi, withManagerApi.
Apply RBAC to sensitive routes	src/app/api/settings/route.ts	Wrap with withApiMiddleware requiring admin role.
Apply RBAC to sensitive routes	src/app/api/admin/ai-usage/route.ts	Wrap with withAdminApi middleware.
Apply RBAC to sensitive routes	src/app/api/companies/bulk/route.ts	Wrap with withApiMiddleware requiring create permission on Company resource.
Apply RBAC to intelligence routes	src/app/api/intelligence/full-pipeline/route.ts	Wrap with withApiMiddleware. Intelligence is premium data.
Apply RBAC to all mutating routes	src/app/api/companies/[id]/notes/route.ts + 15 more	Systematic: wrap all POST/PUT/DELETE routes with appropriate RBAC.
Add session validation middleware	src/lib/api-middleware.ts	Add validateSession() that checks session expiry, user exists in DB, user is active.

Action	File(s)	Changes
Security audit tests	tests/security-rbac.test.ts (NEW)	Test RBAC enforcement: admin can access settings, sales_rep cannot. Test DEMO_MODE disabled in production.

Table 7: Milestone 3 - File-Level Deliverables

Deliverable: DEMO_MODE bypass removed/guarded. All mutating API routes protected with RBAC. DB-level session validation on all protected routes. New security tests pass.

Acceptance Criteria: (1) DEMO_MODE=true does not bypass auth in production build. (2) Unauthenticated POST to /api/settings returns 401. (3) sales_rep role cannot access /api/admin/*. (4) Session validation checks user exists in DB and is active. (5) All existing tests still pass.

Milestone 4: Hybrid Scoring Consolidation

Track: Foundation | Depends on: M1, M2 | Estimated Effort: 2 days

Consolidate the 3 competing scoring systems into the agreed hybrid model: AccountPriorityScore (ICP-fit, deterministic from src/lib/account-prioritization/engine.ts) + IntelligenceScore (evidence-driven, AI-powered from src/lib/engines/scoring-engine.ts). The legacy revenue-intelligence/account-scoring.ts (413 lines, 5 dimensions) must be deprecated. The 6 competing scoring subsystems across the codebase must be mapped and either routed to the correct engine or removed.

Action	File(s)	Changes
Deprecate legacy scorer	src/lib/revenue-intelligence/account-scoring.ts	Add @deprecated JSDoc. Redirect calls to account-prioritization/engine.ts (AccountPriorityScore).
Deprecate legacy scorer	src/lib/revenue-intelligence/account-scorer.ts	Add @deprecated. Merge useful logic into the hybrid model.
Map all scoring call sites	src/app/api/companies/[id]/score/route.ts	Update to call both AccountPriorityScore and IntelligenceScore, return both in response.
Map all scoring call sites	src/app/api/ai/score-leads/route.ts	Route to hybrid scoring. Return AccountPriorityScore for ICP, IntelligenceScore for AI.
Map all scoring call sites	src/app/api/ai/score-contacts/route.ts	Same hybrid routing pattern.
Map all scoring call sites	src/app/api/leads/recalculate-scores/route.ts	Recalculate both scores. Update Prisma Company record with both fields.
Add hybrid scoring types	src/lib/types.ts	Add AccountPriorityScore and IntelligenceScore types. Add HybridScoreResult type.
Update scoring engine output	src/lib/engines/scoring-engine.ts	Add method to produce IntelligenceScore that is clearly separate from AccountPriorityScore.
Consolidation tests	tests/hybrid-scoring.test.ts (NEW)	Test that both scores are computed, returned, and stored correctly.

Table 8: Milestone 4 - File-Level Deliverables

Deliverable: Two-score hybrid model working end-to-end. Legacy scorer deprecated. All API routes return both AccountPriorityScore and IntelligenceScore. Tests pass.

Milestone 5: Enterprise Reasoning Pipeline

Track: Core Intelligence | Depends on: M1, M2 | Estimated Effort: 4-5 days

Wire the Enterprise Reasoning Engine (30-step cumulative reasoning chain) into a callable API pipeline. The engine exists (667 lines, now type-safe after M1) but is not connected to any API route. This milestone creates the integration layer between the engine and the REST API, adds the ReasoningContext persistence layer, and wires it to the command center and company workspace UI components. This is the strategic differentiator of DeepMindQ and must work end-to-end.

Action	File(s)	Changes
Create pipeline integration	src/lib/reasoning-pipeline.ts (NEW)	New file: orchestrateEnterpriseReasoning(companyId). Builds/loads ReasoningContext, runs 30 steps, persists results, returns summary.
Wire to API route	src/app/api/reasoning/route.ts	Enhance existing /api/reasoning to call reasoning-pipeline.ts. Accept companyId, trigger pipeline, return progress/status.
Add reasoning status endpoint	src/app/api/reasoning/[companyId]/status/route.ts (NEW)	New endpoint: GET reasoning status for a company. Return step progress, current phase, last updated.
Wire to UI - Command Center	src/components/intelligence-os/command-center.tsx	Add "Run Reasoning" action button. Show reasoning progress. Display reasoning output in Intelligence Briefing panel.
Wire to UI - Company Workspace	src/components/intelligence-os/company-workspace.tsx	Add reasoning status section. Show 30-step progress, cumulative context, derived outputs.
Add reasoning to store	src/lib/store.ts	Add reasoningStatus, activeReasoningCompanyId to AppState.
Add reasoning to nav	src/lib/nav-config.ts	Add "Intelligence Reasoning" under INTELLIGENCE section (already exists as screen-map entry).
End-to-end test	tests/e2e-reasoning-pipeline.test.ts (NEW)	Test: trigger reasoning for a company, verify all 30 steps execute, verify ReasoningContext/Step persisted.

Table 9: Milestone 5 - File-Level Deliverables

Deliverable: Enterprise reasoning pipeline callable from API. UI triggers and displays reasoning progress. ReasoningContext and ReasoningStep records persisted in DB. End-to-end test passes.

Milestone 6: Multi-Agent Orchestration API

Track: Core Intelligence | Depends on: M1, M2, M5 | Estimated Effort: 3-4 days

Wire the Multi-Agent Orchestrator (10 specialist agents with dependency graph, 413 lines) into the API. The orchestrator exists and is type-safe (after M1) and governance-wired (after M2), but has no API endpoint. This milestone creates the orchestration API, adds agent progress tracking, and connects it to the company workspace UI. The orchestrator should be callable independently or as part of the reasoning pipeline from M5.

Action	File(s)	Changes
Wire to API route	src/app/api/orchestration/route.ts	Enhance existing /api/orchestration to call multi-agent-orchestrator.ts. Accept companyId, return agent execution results.
Add orchestration status	src/app/api/orchestration/[companyId]/status/route.ts (NEW)	GET status: which agents have run, dependency graph progress, total AI calls made.
Integrate with reasoning pipeline	src/lib/reasoning-pipeline.ts	Option to use multi-agent orchestration for steps 19-24 (Internal Intelligence Fusion phase).
Wire to UI	src/components/intelligence-os/company-workspace.tsx	Add agent execution panel. Show 10 agents, their status, dependency graph, AI call count.

Action	File(s)	Changes
Add agent models to schema (if needed)	prisma/schema.prisma	Verify AgentOrchestration model has all fields needed: agentType, status, input, output, tokenCount.
Orchestration tests	tests/e2e-orchestration.test.ts (NEW)	Test: trigger orchestration, verify 3-6 AI calls (not 20+), verify AgentOrchestration records.

Table 10: Milestone 6 - File-Level Deliverables

Milestone 7: Fusion Engine + Continuous Learning

Track: Core Intelligence | Depends on: M1, M5 | Estimated Effort: 3 days

Wire the Fusion Engine (276 lines, pure logic, zero AI calls) and Continuous Learning Loop (204 lines) into the API and UI. The Fusion Engine combines external intelligence with internal capabilities to produce FusionResult records with reasoning chains. The Learning Loop records learning events from wins, losses, feedback, email replies, and meeting notes. Both are type-safe (M1) and should be callable from the company workspace and command center.

Action	File(s)	Changes
Wire fusion to API	src/app/api/fusion/route.ts	Enhance existing /api/fusion. Call fusion-engine.ts with companyId. Return FusionResult[] with reasoning chains.
Wire learning to API	src/app/api/learning/route.ts	Enhance existing /api/learning. Accept learning events: eventType, source, outcome, companyId.
Add feedback collection to UI	src/components/recommendation-feedback-form.tsx	Wire form submission to /api/learning. Convert user feedback into LearningEvent records.
Add fusion results to UI	src/components/intelligence-os/company-workspace.tsx	Add "Opportunity Intelligence" section showing FusionResult items with confidence scores.
Add learning insights to UI	src/components/intelligence-os/intelligence-briefing.tsx	Show "Learning Insights" panel: patterns learned, scoring adjustments, knowledge improvements.
Fusion + Learning tests	tests/e2e-fusion-learning.test.ts (NEW)	Test: trigger fusion, verify FusionResult records. Test: submit feedback, verify LearningEvent created.

Table 11: Milestone 7 - File-Level Deliverables

Milestone 8: Legacy Scoring Cleanup + Remaining @ts-nocheck

Track: Core Intelligence | Depends on: M4 | Estimated Effort: 2-3 days

After M4 deprecated the legacy scorer, this milestone completes the cleanup: remove remaining @ts-nocheck from all 20 non-core files, fix remaining "as any" type escapes in intelligence sources and API routes, and clean up the 328 text-[10px] instances (replace with text-[11px] and remove the CSS override hack at globals.css line 438-443). This milestone drives the codebase toward zero type escapes.

Action	File(s)	Changes
Remove remaining @ts-nocheck	20 files (knowledge-ingestion-pipeline, ai-cache-layer, ai-copilot/*, intelligence-sources/*, API routes)	Remove @ts-nocheck, fix type errors. Many will be simple Prisma model reference fixes.
Remove text-[10px] override hack	src/app/globals.css (lines 438-443)	Remove the `.text-[10px] { font-size: 11px !important }` override.

Action	File(s)	Changes
Replace text-[10px] with text-[11px]	22 files with 328 instances	Find and replace text-[10px] with text-[11px] across all 22 files.
Fix remaining as any	src/lib/research-engine/signal-sequence-engine.ts, opportunity-recommendation.ts, intelligence-health.ts	Replace `as any` casts with proper types.
Fix as any in data-intelligence	src/lib/data-intelligence/engine.ts (line 157)	Replace column mapping `as any` with proper ColumnMapping type.
Clean up legacy revenue-intelligence imports	src/lib/revenue-intelligence/index.ts	Remove deprecated scorer exports. Update to export hybrid scoring functions.
Cleanup verification	scripts/verify-type-safety.sh (NEW)	Script that greps for @ts-nocheck, `as any`, text-[10px] and reports count. Add to CI.

Table 12: Milestone 8 - File-Level Deliverables

Deliverable: Zero @ts-nocheck files. Zero text-[10px] instances. <50 `as any` remaining (only in genuinely unavoidable cases). Legacy scorer fully removed from active imports.

Acceptance Criteria: (1) grep -r "@ts-nocheck" src/ returns 0 matches. (2) grep -r "text-\[10px\]" src/ returns 0 matches. (3) `as any` count < 50. (4) All tests pass.

Milestone 9: Intelligence API Guard + Cost Governance

Track: Core Intelligence | Depends on: M2 | Estimated Effort: 2 days

Implement the Intelligence API Guard (src/lib/intelligence-api-guard.ts) to enforce rate limits, cost budgets, and quality checks on all AI-powered API endpoints. Wire the AI Cost Governance module (src/lib/intelligence-sources/ai-cost-governance.ts, currently @ts-nocheck) into the ModelRouter. This ensures no single company can exhaust AI budget and provides visibility into per-company AI spend.

Action	File(s)	Changes
Wire API guard to AI routes	src/lib/intelligence-api-guard.ts	Add guardAIRequest() function: check rate limit, check cost budget, check quality threshold.
Apply guard to all AI routes	src/app/api/ai/*.ts (15+ routes)	Wrap each AI route handler with guardAIRequest(). Reject with 429 if budget exceeded.
Wire cost governance to ModelRouter	src/lib/engines/model-router.ts	Add cost tracking per company per day/week/month. Log to AICallLog with cost fields.
Enable cost governance	src/lib/intelligence-sources/ai-cost-governance.ts	Remove @ts-nocheck. Wire into ModelRouter. Add budget config to settings.
Add cost dashboard	src/components/screens/ai-usage-dashboard-screen.tsx	Show per-company AI spend, token usage, cost trends. Admin-only view.
Cost governance tests	tests/ai-cost-governance.test.ts (NEW)	Test budget enforcement: after limit reached, AI calls return budget_exceeded status.

Table 13: Milestone 9 - File-Level Deliverables

Milestone 10: Multi-Tenant Schema Foundation

Track: Enterprise Readiness | Depends on: M3 | Estimated Effort: 3-4 days

Add multi-tenancy to the Prisma schema by adding tenantId to all core models. Create a Tenant model, add tenant isolation middleware, and update all queries to filter by tenant. This is the

foundation for enterprise deployment. The current schema has 96 models but none have tenantId. This milestone adds tenantId to the ~40 most critical models (Company, Contact, Opportunity, User, Signal, etc.) and creates the migration strategy.

Action	File(s)	Changes
Add Tenant model	prisma/schema.prisma	Add Tenant model (id, name, slug, settings, createdAt, updatedAt). Add tenantId to User model.
Add tenantId to core models	prisma/schema.prisma	Add tenantId to Company, Contact, Opportunity, Signal, IntelligenceSignal, Note, Draft, Sequence, CapabilityAsset, KnowledgeDocument, AICallLog, ReasoningContext, FusionResult, LearningEvent (~20 models).
Create tenant middleware	src/lib/tenant-middleware.ts (NEW)	Extract tenant from session/jwt. Add to Prisma query context. Provide getCurrentTenant() helper.
Update Prisma client extension	src/lib/db.ts	Add Prisma client extension that auto-filters by tenantId on all queries.
Update proxy.ts for tenant	src/proxy.ts	After auth check, resolve tenant from session. Add tenantId to request context.
Create migration script	scripts/migrate-multi-tenant.ts (NEW)	Script to add tenantId columns, create default tenant, backfill existing data.
Seed data update	scripts/seed.ts	Add default tenant to seed data. All seeded records belong to default tenant.
Tenant isolation tests	tests/multi-tenant.test.ts (NEW)	Test: Company created in tenant A not visible in tenant B queries.

Table 14: Milestone 10 - File-Level Deliverables

Milestone 11: Enterprise Audit Trail + Compliance

Track: Enterprise Readiness | Depends on: M3, M10 | Estimated Effort: 2-3 days

Build a comprehensive audit trail for all sensitive operations (data export, user management, AI configuration, bulk operations). The existing audit-logger.ts provides 11 categories and 3 severity levels but is only used for auth failures. Extend it to cover all CRUD operations on sensitive resources and add a compliance framework for GDPR/data retention.

Action	File(s)	Changes
Extend audit to all mutations	src/lib/audit-logger.ts	Add audit CRUD helper: wraps Prisma operations with audit logging. Add data_access, data_export, data_delete categories.
Apply audit to company mutations	src/app/api/companies/route.ts, companies/[id]/route.ts	Add auditCreate, auditUpdate, auditDelete calls around all mutating operations.
Apply audit to user management	src/app/api/auth/register/route.ts, auth/change-password/route.ts	Audit all user management operations.
Apply audit to AI config changes	src/app/api/settings/route.ts, prompt-templates/route.ts	Audit changes to AI configuration, prompt templates, governance settings.
Create audit dashboard	src/components/screens/audit-screen.tsx	Enhance existing screen: filter by category, severity, user, date range. Export audit logs.
Add data retention config	src/app/api/compliance/route.ts	Enhance existing compliance endpoint: add data retention policies per resource type.

Action	File(s)	Changes
Compliance tests	tests/audit-compliance.test.ts (NEW)	Test: create company, verify audit log entry. Test: export data, verify audit entry.

Table 15: Milestone 11 - File-Level Deliverables

Milestone 12: Data Intelligence + Quality Monitoring

Track: Enterprise Readiness | Depends on: M10 | Estimated Effort: 2-3 days

Integrate the data-intelligence modules (quality-scorer, deduplicator, normalizer, validator, correction-suggester) into an automated monitoring pipeline. These modules exist in src/lib/data-intelligence/ but are not wired to any automated process. Create scheduled data quality scans, automatic duplicate detection, and a data health dashboard that shows quality scores per entity type.

Action	File(s)	Changes
Create quality monitoring service	src/lib/data-quality-monitor.ts (NEW)	Schedule periodic scans: run quality-scorer on all companies, contacts. Store results in DataQualityReport.
Wire dedup to import pipeline	src/app/api/imports/route.ts	Run deduplicator during import. Flag duplicates for review instead of silently deduping.
Wire normalizer to enrichment	src/app/api/companies/enrich/route.ts	Run normalizer on enriched data before storing.
Enhance data health dashboard	src/components/screens/data-health-screen.tsx	Show quality scores per entity, duplicate count, normalization issues, correction suggestions.
Add quality API endpoints	src/app/api/data-health/route.ts	Add endpoints: GET quality summary, POST trigger scan, GET duplicates, POST resolve duplicate.
Data quality tests	tests/data-quality.test.ts (NEW)	Test: trigger scan, verify quality scores stored. Test: import with duplicates, verify flagged.

Table 16: Milestone 12 - File-Level Deliverables

Milestone 13: Screen Consolidation + Store Cleanup

Track: Experience | Depends on: M1-M4 | Estimated Effort: 3-4 days

Consolidate 80 screen components into approximately 20 primary views. The current store.ts has 63 view IDs that should be reduced to approximately 20. Many screens overlap in functionality (e.g., companies-screen.tsx and company-profile-screen.tsx and company-detail-screen.tsx serve overlapping purposes). Remove 4 .bak files. Update screen-map.tsx, nav-config.ts, and store.ts to reflect the consolidated view structure. This directly addresses designer critique points about navigation complexity.

Action	File(s)	Changes
Consolidate store views	src/lib/store.ts	Reduce ViewId from 63 to ~20. Remove legacy aliases. Keep: command-center, accounts, company-workspace, knowledge-workspace, capability-workspace, intelligence-search, import, analytics, settings, data-health, ai-health, audit, company-detail, contact-detail.

Action	File(s)	Changes
Update screen map	src/lib/screen-map.tsx	Remove ~40 unused lazy imports. Map legacy view IDs to consolidated views.
Update navigation	src/lib/nav-config.ts	Simplify to 3 sections with ~15 total items. INTELLIGENCE: 3 items. WORKSPACES: 3 items. ADMIN: 9 items.
Merge duplicate screens	src/components/screens/companies-screen.tsx + company-profile-screen.tsx	Merge into single AccountsScreen with detail view. Remove company-profile-screen.tsx.
Remove .bak files	src/components/screens/*.bak, *.bak2 (4 files)	Delete: contact-detail-screen.tsx.bak2, company-detail-screen.tsx.bak, company-detail-screen.tsx.bak2, opportunity-workspace-screen.tsx.bak2.
Consolidate intelligence screens	src/components/screens/signal-intelligence-screen.tsx, internal-intelligence-screen.tsx, account-intelligence-screen.tsx	Merge into unified IntelligenceScreen accessible from company workspace.
Update Intelligence OS screens	src/components/intelligence-os/*.tsx (7 files)	Verify all 7 Intelligence OS screens work with consolidated view IDs. Update imports.
Update app-shell routing	src/components/app-shell.tsx	Update view routing to use consolidated view IDs. Remove legacy fallbacks.
Consolidation tests	tests/screen-consolidation.test.ts (NEW)	Test: all 20 primary views render without errors. Test: no 404s on valid view IDs.

Table 17: Milestone 13 - File-Level Deliverables

Milestone 14: Design System Implementation

Track: Experience | Depends on: M13 | Estimated Effort: 4-5 days

Implement the 14-point UI/UX design improvements identified in the designer debate. Key deliverables: (1) Replace generic EmptyState with contextual empty states per view type. (2) Add side panel for AI brief (designer answer Q1). (3) Implement single AI draft + editable output (Q2). (4) Add "Unverified Signals" section (Q3). (5) Make Workbench/Governance admin-only (Q4). (6) Implement choreographed animations with prefers-reduced-motion respect (Q5). This milestone transforms the UI from functional to enterprise-grade.

Action	File(s)	Changes
Contextual empty states	src/components/shared/design-system.tsx	Replace generic EmptyState with createCompanyEmpty, createContactEmpty, createSignalEmpty, etc. Each with relevant icon, description, and CTA.
Side panel for AI brief	src/components/shared/ai-chat-sidebar.tsx	Convert from floating chat to docked side panel (320px). Shows AI brief, reasoning progress, action suggestions. Toggle from command bar.
Single draft + editable	src/components/screens/drafts-screen.tsx	Show single AI-generated draft with inline editing. "Regenerate" button. No multiple drafts.
Unverified Signals section	src/components/intelligence-os/company-workspace.tsx	Add "Unverified Signals" panel: signals below confidence threshold, shown separately with "Verify" action.
Admin-only workbench	src/lib/nav-config.ts	Add admin-only flag to: AI Health, Governance Workbench, System Health. Only visible to admin role users.

Action	File(s)	Changes
Choreographed animations	src/app/globals.css	Add entrance animations: fadeSlideUp, stagger children. Respect prefers-reduced-motion (already at line 409).
Animation choreography	src/components/intelligence-os/command-center.tsx	Apply staggered entrance to cards. Add loading skeleton with shimmer.
Accessibility refinements	src/components/shared/enterprise-components.tsx	Complete WCAG: focus management, keyboard navigation, screen reader announcements for dynamic content.
Design QA tests	tests/design-system.test.tsx	Extend existing tests: verify contextual empty states, verify admin-only visibility, verify animation class application.

Table 18: Milestone 14 - File-Level Deliverables

Milestone 15: Enterprise SSO + User Management

Track: Experience | Depends on: M10, M11 | Estimated Effort: 3 days

Add enterprise SSO support (SAML/OIDC) and proper user management to the platform. The current auth system uses OTP-based email auth which works for single-instance deployment but must be extended for enterprise. Add user invitation workflow, role assignment, and session management for enterprise environments.

Action	File(s)	Changes
Add SSO support	src/lib/sso.ts (NEW)	Implement SAML/OIDC authentication flow. Support IdP-initiated and SP-initiated SSO.
Add SSO API routes	src/app/api/auth/sso/callback/route.ts (NEW)	Handle SSO callback. Exchange token, create/update user session.
Add user invitation	src/app/api/auth/invite/route.ts (NEW)	Admin invites users by email. Send invitation link. Assign role on signup.
Enhance user management UI	src/components/screens/settings-screen.tsx	Add "Team" tab: invite users, assign roles, revoke access, view active sessions.
Add role management	src/lib/rbac.ts	Add ability to create custom roles (beyond admin/manager/sales_rep). Add Role model to Prisma if needed.
Session management	src/lib/session.ts	Add session listing, session revocation, concurrent session limits.
SSO tests	tests/sso-auth.test.ts (NEW)	Test: SSO callback creates session. Test: invited user can signup with role.

Table 19: Milestone 15 - File-Level Deliverables

Milestone 16: Predictive Intelligence + Business BI

Track: Integration | Depends on: M5, M7, M12 | Estimated Effort: 3-4 days

Build the Business Intelligence Layer (Layer 5) with predictive modeling capabilities. The current pipeline-screen.tsx and pipeline-forecast-screen.tsx exist but use basic statistical projections. This milestone adds AI-powered predictive models using the reasoning engine outputs: win probability prediction, deal velocity forecasting, churn risk scoring, and revenue forecasting. The ScoringEngine (9 dimensions) provides the foundation, but predictions must incorporate reasoning engine outputs and learning loop adjustments.

Action	File(s)	Changes
Create prediction engine	src/lib/prediction-engine.ts (NEW)	AI-powered predictions using reasoning context + historical data + learning loop weights. Methods: predictWinProbability, forecastRevenue, estimateDealVelocity, scoreChurnRisk.
Wire to pipeline forecast	src/app/api/pipeline/forecast/route.ts	Enhance to use prediction engine instead of basic projections.
Wire to opportunity scoring	src/app/api/opportunities/route.ts	Add predicted win probability from reasoning context.
Update pipeline health screen	src/components/screens/pipeline-health-screen.tsx	Show AI predictions alongside manual assessments. Confidence scores on predictions.
Update deal coaching	src/components/screens/deal-coaching-screen.tsx	Add AI-generated deal coaching based on reasoning context and historical patterns.
Prediction accuracy tracking	src/lib/continuous-learning-loop.ts	Track prediction accuracy: record predicted vs actual outcomes. Use for model calibration.
Prediction tests	tests/prediction-engine.test.ts (NEW)	Test: prediction returns confidence-bounded result. Test: accuracy tracking records outcomes.

Table 20: Milestone 16 - File-Level Deliverables

Milestone 17: Knowledge Graph + Intelligent Retrieval

Track: Integration | Depends on: M5, M7, M12 | Estimated Effort: 3 days

Enhance the RetrievalEngine and knowledge infrastructure to create a true organizational knowledge graph. The current RetrievalEngine (484 lines) provides local embeddings for knowledge search. The KnowledgeDocument model exists in Prisma. The knowledge-ingestion-pipeline.ts exists but is @ts-nocheck. This milestone completes the knowledge pipeline: document ingestion, embedding generation, cross-company knowledge transfer, and intelligent retrieval that surfaces relevant case studies, proof points, and capabilities for each deal.

Action	File(s)	Changes
Complete knowledge ingestion	src/lib/knowledge-ingestion-pipeline.ts	Remove @ts-nocheck. Fix types. Wire to KnowledgeDocument model. Support PDF, DOCX, text ingestion.
Enhance retrieval	src/lib/engines/retrieval-engine.ts	Add cross-company knowledge retrieval: "find similar deals", "find relevant case studies", "find capability proof points".
Add knowledge search API	src/app/api/knowledge/graph/route.ts	Enhance to support semantic search across all knowledge types. Return ranked results with relevance scores.
Wire to company workspace	src/components/intelligence-os/company-workspace.tsx	Add "Knowledge" section: auto-surfaced relevant case studies, capability proof points based on reasoning context.
Wire to knowledge workspace	src/components/intelligence-os/knowledge-workspace.tsx	Add document management: upload, tag, search, link to companies/opportunities.
Knowledge tests	tests/knowledge-pipeline.test.ts (NEW)	Test: ingest document, verify KnowledgeDocument created. Test: search returns relevant results.

Table 21: Milestone 17 - File-Level Deliverables

Milestone 18: Enterprise Deployment + Final Integration

Track: Polish | Depends on: All previous | Estimated Effort: 3-4 days

Final integration milestone: ensure all 8 layers work together, deploy to production infrastructure, and validate end-to-end flows. This includes: finalizing Docker deployment with multi-tenant support, setting up monitoring and alerting, running comprehensive E2E tests across the full intelligence pipeline, validating security controls under penetration testing, and producing the final deployment documentation.

Action	File(s)	Changes
Update Docker config	Dockerfile, docker-compose.yml	Add multi-tenant env vars. Add health checks for reasoning pipeline. Update backup script.
Update next.config.ts	next.config.ts	Ensure output: standalone is set. Add security headers. Configure CSP for production domains.
E2E intelligence flow test	tests/e2e-full-intelligence.test.ts (NEW)	Test complete flow: import company > enrich > reason > score > generate brief > fusion > learning. Verify all DB records created.
Security validation	tests/security-penetration.test.ts (NEW)	Test: SQL injection, XSS, CSRF, unauthorized access, tenant isolation bypass attempts.
Performance benchmarks	scripts/benchmark-intelligence.ts (NEW)	Benchmark: reasoning pipeline time, AI call count, concurrent company processing.
Update CI/CD	.github/workflows/ci.yml	Add E2E tests, security tests, performance benchmarks to CI pipeline.
Deployment runbook	docs/deployment-runbook.md (NEW)	Step-by-step production deployment guide: env vars, DB migration, seed data, monitoring setup.
Final quality gates	error-snapshots/baseline-final.json (NEW)	Final baseline: TSC 0, ESLint 0, 1000+ tests pass, build success, security tests pass, E2E tests pass.

Table 22: Milestone 18 - File-Level Deliverables

Chapter 5: Milestone Dependency Chain

The dependency chain is structured to maximize parallelism while respecting technical dependencies. Foundation milestones (M1-M4) are sequential and must complete before parallel work begins. Core Intelligence milestones (M5-M9) can partially parallelize: M5 and M7 can run concurrently after M1+M2. M6 requires M5. M9 requires M2 only. Enterprise Readiness (M10-M12) requires M3 (security) as foundation. Experience (M13-M15) requires M13 first, then M14+M15 in parallel. Integration (M16-M18) requires relevant core milestones.

Milestone	Depends On	Can Parallel With
M1: Type Safety	None	Nothing (start)
M2: Governance Wiring	M1	M3 (both need M1)
M3: Security Hardening	M1	M2 (both need M1)
M4: Hybrid Scoring	M1, M2	M8

Milestone	Depends On	Can Parallel With
M5: Reasoning Pipeline	M1, M2	M7 (both need M1+M2)
M6: Agent Orchestration	M1, M2, M5	Nothing (needs M5 output)
M7: Fusion + Learning	M1, M5	M5 (both need M1)
M8: Legacy Cleanup	M4	M9, M10
M9: Cost Governance	M2	M8, M10
M10: Multi-Tenant Schema	M3	M8, M9, M11
M11: Audit + Compliance	M3, M10	M12
M12: Data Quality	M10	M11
M13: Screen Consolidation	M1-M4	M14
M14: Design System	M13	M15
M15: SSO + User Mgmt	M10, M11	M14
M16: Predictive BI	M5, M7, M12	M17
M17: Knowledge Graph	M5, M7, M12	M16
M18: Final Integration	All	Nothing (end)

Table 23: Milestone Dependency Matrix

Chapter 6: Risk Register

Risk	Impact	Likelihood	Mitigation
@ts-nocheck removal reveals deeper type incompatibilities	High - blocks M2-M9	Medium	M1 is designed as a focused milestone. If types are severely broken, create interface adapters instead of fixing all types.
Governance wiring adds latency to every AI call	Medium - degraded UX	Medium	Governance checks are pre-generation only (no LLM call). Measured at <50ms. Cache governance results for same context.
Multi-tenant migration breaks existing data	Critical - data loss	Low	M10 includes backfill script and default tenant. Run migration on backup first. Migration is additive (adds columns, not removes).
Screen consolidation breaks user workflows	High - user frustration	Medium	Keep legacy view ID aliases in screen-map.tsx. Log usage of legacy views before removing.
Reasoning pipeline too slow for interactive use	Medium - poor UX	Medium	Run pipeline async. Show progress in UI. Cache results. Target <30 seconds for full 30-step pipeline.
Enterprise SSO integration complexity	Medium - delays M15	Low	Use established library (next-auth SAML provider). Start with OIDC (simpler), add SAML later.

Risk	Impact	Likelihood	Mitigation
Scope creep in design system (M14)	Medium – delays later milestones	High	M14 is explicitly scoped to 14 designer points. No additional features. Defer to M18 for polish.

Table 24: Risk Register

Chapter 7: New Files Creation Summary

Across all 18 milestones, approximately 25 new files will be created. This section provides a consolidated view of all new files, organized by type. Existing files that are modified (not listed here) are detailed in the milestone tables above.

File Path	Created In	Purpose
tests/enterprise-reasoning.test.ts	M1	Type validation tests for reasoning engine
tests/multi-agent-orchestrator.test.ts	M1	Type validation tests for orchestrator
tests/governance-wiring.test.ts	M2	Verify all engines use governance before LLM calls
tests/security-rbac.test.ts	M3	RBAC enforcement and DEMO_MODE tests
tests/hybrid-scoring.test.ts	M4	Two-score hybrid model tests
src/lib/reasoning-pipeline.ts	M5	Enterprise reasoning pipeline orchestration
src/app/api/reasoning/[companyId]/status/route.ts	M5	Reasoning status API endpoint
tests/e2e-reasoning-pipeline.test.ts	M5	End-to-end reasoning pipeline test
src/app/api/orchestration/[companyId]/status/route.ts	M6	Orchestration status endpoint
tests/e2e-orchestration.test.ts	M6	End-to-end orchestration test
tests/e2e-fusion-learning.test.ts	M7	Fusion engine + learning loop tests
scripts/verify-type-safety.sh	M8	CI script to verify zero @ts-nocheck and as any
tests/ai-cost-governance.test.ts	M9	Cost governance enforcement tests
src/lib/tenant-middleware.ts	M10	Multi-tenant isolation middleware
scripts/migrate-multi-tenant.ts	M10	Schema migration script for tenant support
tests/multi-tenant.test.ts	M10	Tenant isolation tests
tests/audit-compliance.test.ts	M11	Audit trail and compliance tests
src/lib/data-quality-monitor.ts	M12	Automated data quality monitoring service
tests/data-quality.test.ts	M12	Data quality pipeline tests
tests/screen-consolidation.test.ts	M13	Screen consolidation tests
src/lib/sso.ts	M15	SAML/OIDC SSO authentication
src/app/api/auth/sso/callback/route.ts	M15	SSO callback handler
src/app/api/auth/invite/route.ts	M15	User invitation endpoint
tests/sso-auth.test.ts	M15	SSO and invitation workflow tests

File Path	Created In	Purpose
src/lib/prediction-engine.ts	M16	AI-powered predictive intelligence
tests/prediction-engine.test.ts	M16	Prediction accuracy and calibration tests
tests/knowledge-pipeline.test.ts	M17	Knowledge ingestion and retrieval tests
tests/e2e-full-intelligence.test.ts	M18	Full E2E intelligence pipeline test
tests/security-penetration.test.ts	M18	Security penetration tests
scripts/benchmark-intelligence.ts	M18	Intelligence pipeline benchmarks
docs/deployment-runbook.md	M18	Production deployment guide
error-snapshots/baseline-final.json	M18	Final quality baseline snapshot

Table 25: Complete New Files Registry

Chapter 8: Modified Files Summary

The following table consolidates all existing files that will be modified across the 18 milestones, organized by architectural layer. Each file may be modified in multiple milestones as the plan progresses.

Existing File	Modified In	Nature of Change
src/lib/enterprise-reasoning-engine.ts	M1, M2	Remove @ts-nocheck, wire governance
src/lib/multi-agent-orchestrator.ts	M1, M2	Remove @ts-nocheck, wire governance
src/lib/fusion-engine.ts	M1	Remove @ts-nocheck, validate types
src/lib/continuous-learning-loop.ts	M1	Remove @ts-nocheck, validate types
src/lib/engines/synthesis-engine.ts	M2	Wire governance before LLM calls
src/lib/engines/scoring-engine.ts	M2, M4	Wire governance, add IntelligenceScore
src/lib/engines/action-engine.ts	M2	Wire governance before LLM calls
src/lib/engines/conversation-engine.ts	M2	Wire governance before LLM calls
src/lib/engines/retrieval-engine.ts	M2, M17	Wire governance, enhance retrieval
src/lib/engines/model-router.ts	M9	Add cost tracking per company
src/proxy.ts	M3, M10	Remove DEMO_MODE, add tenant resolution
src/lib/auth-helpers.ts	M3	Enhance withApiMiddleware with role support
src/lib/rbac.ts	M3, M15	Add custom roles for enterprise
src/lib/revenue-intelligence/account-scoring.ts	M4, M8	Deprecate, then remove
src/lib/store.ts	M5, M13	Add reasoning state, consolidate views
src/lib/nav-config.ts	M5, M13, M14	Add reasoning nav, simplify, admin-only flags
src/lib/screen-map.tsx	M13	Remove ~40 unused lazy imports

Existing File	Modified In	Nature of Change
src/app/globals.css	M8, M14	Remove text-[10px] hack, add animations
prisma/schema.prisma	M10	Add Tenant model, tenantId to ~20 models
src/lib/db.ts	M10	Add Prisma client extension for tenant filtering
src/lib/audit-logger.ts	M11	Add CRUD audit helpers
src/lib/data-intelligence/engine.ts	M8, M12	Fix as any, wire to monitoring
src/lib/knowledge-ingestion-pipeline.ts	M8, M17	Remove @ts-nocheck, complete pipeline
src/components/intelligence-os/company-workspace.tsx	M5, M6, M7, M17	Add reasoning/fusion/knowledge sections
src/components/intelligence-os/command-center.tsx	M5, M14	Add reasoning trigger, animations
src/components/screens/data-health-screen.tsx	M12	Add quality monitoring dashboard
Dockerfile + docker-compose.yml	M18	Multi-tenant support, updated health checks
next.config.ts	M18	Standalone output, production headers

Table 26: Existing Files Modified Across Milestones

Chapter 9: Effort Estimate

Track	Milestones	Days (Sequential)	Days (Parallelized)
Foundation	M1, M2, M3, M4	9-11 days	6-8 days
Core Intelligence	M5, M6, M7, M8, M9	14-17 days	10-12 days
Enterprise Readiness	M10, M11, M12	8-10 days	5-7 days
Experience	M13, M14, M15	10-12 days	7-9 days
Integration + Polish	M16, M17, M18	9-11 days	7-9 days
TOTAL	M1-M18	50-61 days	35-45 days

Table 27: Effort Estimate by Track

The total effort ranges from 35 to 45 working days with maximum parallelization, or 50 to 61 days if executed sequentially. With a single developer and 2-day buffer per track, the realistic timeline is approximately 50-55 working days (10-11 weeks). With 2 developers working in parallel on independent tracks, the timeline can be compressed to approximately 6-7 weeks. The critical path runs through M1 > M2 > M5 > M6 > M18.

This plan is submitted for debate and review. No coding should begin until stakeholders agree on the milestone sequence, scope, and acceptance criteria. Each milestone should be treated as a standalone deliverable that can be verified independently. The dependency chain allows for parallel execution on independent tracks after the foundation milestones (M1-M4) are complete.